Object-Oriented Programming I

Dr. Ghemougui Abdessettar

University of M'Sila – Faculty of Mathematics and Computer Science

October 14, 2026

Lecture plan

- Object Initialization
- Constructors
- Destructors

Object initialization

If we declare a variable

```
1 int x;
```

what value should we expect in `x`? It is **uninitialized**. It is always advisable to give variables an initial value so that they are in a defined state.

What about our objects?

```
1 Stack s;
```

- What is the value of the attribute `top`?
- What values are stored in the array `mem`?

The need for a constructor

In the procedural version we defined `init`, but the programmer can forget to call it and use an uninitialized stack.

In OOP we always want our objects to be in a **valid state**.

Constructor

A special member function that is called **automatically** when an object is created. Its purpose is to initialize the object. It has the same name as its class and has **no return type**.

Warning

A constructor has no return type, not even `void`.

A first constructor

```
#define CAPACITY 100

class Stack {
private:
  int mem[CAPACITY];
  int top;
public:
  // constructor
  Stack() {
    top = 0;
    for (int i = 0; i < CAPACITY; i++)
      mem[i] = 0;
  }
  // methods
  bool isEmpty() { return top == 0; }
  bool isFull()  { return top == CAPACITY; }
```

Member initializer lists

In C++, constructors can use **member initializer lists**. Here, the attribute `top` is set to 0 *before* the body of the constructor runs.

```
1 // constructor
2 Stack() : top(0) {
3   for (int i = 0; i < CAPACITY; i++)
4     mem[i] = 0;
5 }
```

Like methods, constructors can accept **arguments**.

Example update: a custom capacity

Question

Our `Stack` uses a static capacity defined by the constant `CAPACITY`: all `Stack` objects have the same fixed capacity. What if we want to create stacks with a custom capacity?

- The capacity becomes part of the stack's **state**
- It is defined when the stack is created
- An array has a fixed size, so `mem` must be allocated **dynamically**
- Setting the capacity and allocating memory is the job of the **constructor**
- The constructor takes the capacity as

Stack

Constructor with an argument

```
#include <cstdlib> // malloc and free

class Stack {
private:
  int *mem;
  int top;
  int capacity;
public:
  // constructor: allocates the memory of the stack
  Stack(int capacity) : mem(nullptr), top(0), capacity(capacity) {
    mem = (int *)malloc(capacity * sizeof(int));
    for (int i = 0; i < capacity; i++)
      mem[i] = 0;
  }
  bool isEmpty() { return top == 0; }
  bool isFull()  { return top == capacity; }
```

Creating stacks of different capacities

```
int main() {
  Stack s1(10), s2(20); // two objects with different capacities
  s1.push(10);
  s2.push(5);
  s1.push(8);
  std::cout << "Stack 1 capacity : " << s1.getCapacity() << std::endl;
  std::cout << "Stack 2 capacity : " << s2.getCapacity() << std::endl;
}
```

Note

`malloc` is declared in `<cstdlib>`. The members are initialized in the order in which they are *declared* in the class (`mem`, `top`, `capacity`), not in the order of the initializer list, so we write the list in the same order.

Note

The problem: giving resources back

An object can acquire resources when it is initialized. **How do we ensure it gives them back when it is finished?**

In our last version of `Stack`, the constructor allocates dynamic memory. How does the object release it?

The problem: giving resources back

An object can acquire resources when it is initialized. **How do we ensure it gives them back when it is finished?**

In our last version of `Stack`, the constructor allocates dynamic memory. How does the object release it?

- We cannot call `free` on `mem` from outside: it is part of the state and it is private.

The problem: giving resources back

An object can acquire resources when it is initialized. **How do we ensure it gives them back when it is finished?**

In our last version of `Stack`, the constructor allocates dynamic memory. How does the object release it?

- We cannot call `free` on `mem` from outside: it is part of the state and it is private.
- We cannot make the pointer public: that would violate encapsulation.

The problem: giving resources back

An object can acquire resources when it is initialized. **How do we ensure it gives them back when it is finished?**

In our last version of `Stack`, the constructor allocates dynamic memory. How does the object release it?

- We cannot call `free` on `mem` from outside: it is part of the state and it is private.
- We cannot make the pointer public: that would violate encapsulation.
- We could add a `release()` method, but the programmer may forget to call it, or call it too early and keep using the object after the memory was freed.

Destructors

Destructor

A special method that is called **automatically** when the object is about to be destroyed. We use it to release the resources acquired in the constructor.

- same name as the class, preceded by ~
- no parameters
- no return type

A destructor for Stack

```
class Stack {
private:
  int *mem;
  int top;
  int capacity;
public:
  // constructor
  Stack(int capacity) : mem(nullptr), top(0), capacity(capacity) {
    mem = (int *)malloc(capacity * sizeof(int));
    for (int i = 0; i < capacity; i++)
      mem[i] = 0;
    std::cout << "Stack created\n";
  }
  // destructor
  ~Stack() {
    free(mem);      // free the allocated memory
```

The destructor runs automatically

```
int main() {
  Stack s1(10), s2(20);
  s1.push(5);
  s2.push(8);
  std::cout << "Stack 1 capacity : "
            << s1.getCapacity() << std::
    endl;
  std::cout << "Stack 2 capacity : "
            << s2.getCapacity() << std::
    endl;
} // s2 then s1 are destroyed here,
  // in reverse order of creation
```

Output:

```
Stack created
Stack created
Stack 1 capacity : 10
Stack 2 capacity : 20
Stack destroyed
Stack destroyed
```

Note

A warning for next time

Warning

Our `Stack` owns a raw pointer. Copying a `Stack` (`Stack s3 = s1;`) copies the *pointer*, not the memory. Both objects would then free the same block: a **double free**.

Handling copies correctly is the topic of the next lecture.

Summary

- The procedural stack separates data from operations, does not protect its data, and does not guarantee initialization.
- A **class** bundles **state** (attributes) and **behavior** (methods); we work with its **objects**.
- **Encapsulation** hides the state and protects the object's **invariants**; it is more than "private attributes".
- A **constructor** puts every new object in a valid state and can acquire resources.
- A **destructor** releases those resources automatically.

Course materials



`https://aguemoug.github.io/oop-course/`

Lecture notes, slides, labs, and source code